Typed Scalar Extraction and Observation Boundaries

Mettapedia Working Notes

1 November 2026

Abstract

A proof-producing extractor translates a fragment of existing PeTTa atoms into a result-indexed scalar representation. It handles signed integer and symbol data, Boolean comparisons, conditional branches and flat tuple results. Successful evaluation refines the existing source evaluator, including singleton answer multiplicity, unchanged state and empty I/O. Callback preparation uses the actual source clause selector and captures, while certificate revalidation checks local program dependencies. A runtime counterexample illustrates the central information boundary: Boolean comparisons encoded as integer zero or one are adequate for branch selection but not for ordinary value observation. These are source and representation laws, not a verification of the native C recognizer or the whole fold builtin.

1 A result encoding must name its observers

The native scalar-fold audit found a comparison node producing a register marked as an integer, with value zero or one. The recognizer also admitted comparison nodes as ordinary returned expressions, tuple fields and operands of other scalar operations. On the inspected binary, native and reference routes consequently disagreed:

Observation	Ordinary result	Native scalar result
Returned comparison	Boolean true or false	Integer 1 or 0
Comparison in a tuple	`(tag true)`	`(tag 1)`
Comparison consumed by addition	Caught arithmetic type error	Successful integer 1
Equality of a true comparison with integer 1	False	Integer 1
Comparison used only to choose 7 or 9	7	7

The positive branch control is important. Let $e(b) = 1$ when b is true and $e(b) = 0$ otherwise. Then branching on $e(b) \neq 0$ preserves the selected branch. It does not follow that publishing $e(b)$ preserves the source value. The formalization proves that no function $d : \mathbb{Z} \to \mathsf{Atom}$ can satisfy both

$$\forall n,\ d(n) = \mathsf{integer}(n) \quad \text{and} \quad \forall b,\ d(e(b)) = \mathsf{boolean}(b).$$

The two required meanings of $d(1)$ conflict. A static result sort or runtime tag must retain the distinction. This is an exact information-loss boundary, not a claim about a globally minimum encoding.

2 The typed scalar core

`Machines.TypedScalar` separates two result kinds:

$$\mathsf{Value}(\mathsf{data}) = \mathbb{Z} + \mathsf{String}, \qquad \mathsf{Value}(\mathsf{test}) = \mathsf{Bool}.$$

Register reads return data. Their integer/symbol alternative may vary between executions; the kind system does not require a permanently integer register. Arithmetic requires data operands and dynamically refuses non-integers. Comparisons return tests. Equality can compare either kind and keeps Boolean versus integer values distinct. A conditional requires a test and evaluates only its selected branch.

The core interpreter is parameterized by its register environment. The read footprint theorem proves that agreement on all referenced registers suffices for equal results. This is a conservative static footprint, including both conditional branches, not a minimum dynamic read set. Repeated reads retain one shared environment and therefore cannot acquire independent values.

The result index supplies information that can be erased in some physical positions and must be restored in others. A branch selector can use a Boolean bit. A returned field must be encoded according to its kind. The typed core does not silently allow a predicate to feed integer addition.

3 Extraction from existing source atoms

`PeTTa.Bridges.ScalarSource` consumes the existing `OSLFCore.Atom` carrier. Its supported forms include signed integer literals, inert symbol literals, Boolean literals, captured data variables, addition, subtraction, integer orders, equality and conditional expressions. The extractor returns the inferred result kind, an expression at that kind and a structural elaboration derivation.

The correspondence theorem is proved by induction on that derivation, using the existing PeTTa primitive, argument-continuation and conditional laws. If the extracted expression succeeds with value v, the original expression has a `PureReturns` derivation for the correctly encoded v, with unchanged state and exactly one answer. This is not established by defining the source evaluator to be the extracted interpreter.

The completed-run theorem then characterizes every completed source run: its state is unchanged, its answer list is the singleton encoded value, and its input and output observations are empty. It also establishes existence of sufficient source fuel. It does not supply equality between native work and evaluator transitions, or permission to bypass a source-visible budget.

Unknown variables, effects, nondeterministic answer constructors and invalid sort combinations decline extraction. Decline is not a source-language error: the ordinary evaluator remains responsible for that expression. In particular, Boolean arithmetic declines rather than being silently coerced to integer arithmetic. Both branches are checked during extraction, even though only the selected branch is evaluated.

The source model is the repository's explicit PeTTa operational profile. Primitive heads have its native meaning. The extractor additionally requires that no equation redefine an admitted primitive head. Inert-symbol evidence excludes source-callable heads and control forms. Correspondence with all native profiles, foreign primitive registrations and their revision counters remains an implementation bridge.

4 Flat tuples and live callability

`ScalarBody` compiles scalar results, flat tuples and result-level conditional branches. Tuple fields carry their individual result kinds, so a comparison field materializes a Boolean rather than an integer. The source proof composes the existing left-to-right data-argument continuation; it preserves field order and does not quotient answer occurrences.

Static and live head admission are distinct. A source tuple may start with a variable, an integer or an inert symbol. After its fields are computed, the body evaluator checks the actual head against the captured program. A symbol that now names a callable function declines. The negative control keeps the same variable-headed tuple syntax but adds a definition for its produced head; static body compilation succeeds while live evaluation declines.

This boundary is relevant to caching and lifetime management. The source certificate records its program and capture assumptions, while the produced head still has a live obligation. Private field evaluation cannot justify publishing a partially accepted tuple.

5 Call-specific selection and captured registers

`ScalarInvocation.prepare` uses the existing `Eval.clauses` selector on the supplied accumulator and item values. It requires exactly one selected evaluation body. It then extracts scalar registers from the selected substitution and compiles that body. Registers are justified by the actual lookup and scalar-decoding results, including bindings obtained from tuple patterns.

The main theorem concerns an actual two-variable source call whose arguments are already captured values. An accepted prepared result is that call's singleton answer with unchanged state and no I/O. Primitive/control-head exclusions, registration and source clause selection remain explicit checked conditions.

This permits a useful widening relative to a globally single-equation rule. An additional equation whose argument pattern cannot match the current call does not disable extraction. Two matching answer-producing equations decline. The controls also distinguish a repeated-variable tuple pattern applied to equal versus unequal arguments. These cases use the existing matcher rather than treating tuple holes as independently scoped variables.

No rule here removes an arbitrary applicable equation merely because it is expected to be unproductive. Proving a selected body's empty answer stream would be an additional analysis. Full fold invocation, changing accumulator shapes across iterations and non-single-answer callbacks also require the appropriate surrounding protocol.

6 Local certificate revalidation

`ScalarDependencies` computes the primitive and inert-symbol head requirements of an extracted scalar expression. A transport theorem re-establishes its source elaboration under a new program satisfying those requirements. The executable revalidator returns a certificate indexed by that new program, rather than attaching an old proof to a new revision.

Consequently an unrelated equation need not invalidate a scalar certificate, whereas redefining its comparison primitive or activating a required inert symbol does. These are semantic dependency tests. A cheap revision-stamp implementation needs a separate proof that its stamps cover the same dependencies. The scalar cache law also does not erase the live tuple-head test or the callback's selected-clause obligation.

The resulting region judgment combines several kinds of requisite information:

Evidence	What it preserves
Result kind	Boolean versus integer/symbol interpretation
Captured environment	Shared variable values and tuple-pattern bindings
Program dependencies	The head meanings used by the certificate
Live head check	Data versus callable result interpretation
Completion observation	Singleton result, state and I/O behavior

This is a concrete connection between dependent result types and optimization admission. It does not select a universal Prime calculus or identify every future observer with this source profile.

7 Qualification and remaining bridges

`ScalarExtractionAudit` imports the generic typed core and six source modules, prints the principal axiom dependencies and emits declaration-level foundation manifests. Positive and negative controls cover scalar and tuple Booleans, nested comparisons, symbol arithmetic, effects, repeated captures, callability changes, irrelevant and duplicate clauses, and local revalidation. The seven mathematical modules pass strict warning-as-error checks. Their manifests cover 585 checked declaration bodies, with eight nonlogical recursion companions listed separately, no admissions and no added axioms. Transitive primitive dependencies are limited to `propext`, `Classical.choice` and `Quot.sound`.

The native discrepancy has a separate differential fixture and captured outputs. A successful Lean audit does not repair that C path. Native Boolean tagging or predicate-only admission needs its own regression and mutation qualification.

The next correspondence layer is a bounded, typed register implementation with checked signed arithmetic, correct result materialization and refusal before public effects. The existing private-fold law supplies the restart boundary, but complete builtin invocation, source budget and yield accounting, allocation failure and native lifetime obligations are not discharged by the callback extraction theorem.